📝 Policy Manager Agentic Interface Contract

Status: Draft 01, for review with Veersa and Outseer
Owner: Thoughtful Labs
Scope: Interface between the Thoughtful Labs agentic layer and the Policy Manager chat backend, shared data services and simulation module.

Policy Manager recommends new or improved fraud policies from missed fraud, change points and historical outcomes, then compares them with an existing policy baseline. Thoughtful Labs owns recommendation and explanation. Veersa owns data access, policy execution and metric calculation. Outseer SMEs own business approval.

This draft follows the supplied Case Manager interface contract and Policy Manager PRD. PM-specific routes, schemas, scope names and orchestration below are proposed contracts, not confirmed upstream APIs. Illustrative IDs and results are fixtures. The Case Manager sprint commitments do not establish Policy Manager delivery dates.

1 The three contracts

Contract	Direction	Spec by	Built by
Chat — POST /v1/chat	PM chat backend → agent	Thoughtful Labs	Thoughtful Labs
Agentic policy context — GET /agentic/v1/policy-context	Agent → shared data services	Thoughtful Labs, validated with Veersa	Veersa
Backtesting — POST /agentic/v1/simulations and GET /agentic/v1/simulations/{simulationId}	Agent → simulation module	Joint Thoughtful Labs and Veersa	Veersa

The proposed agent tool loop retrieves context, generates a candidate, submits it for backtesting, retrieves results and explains the comparison. The PM backend owns the user

session, chat integration and SME review workflow. It does not execute a second simulation for the same candidate.

Creating a simulation is permitted. Creating, updating, retiring or deploying a production policy is outside these interfaces. A retirement recommendation is advisory; it is not a retirement command.

2 Topology and auth

Product path: Analyst → existing Policy Manager UI → PM chat backend → Agent → Agentic API → shared data services / simulation module.

Sandbox path: Sandbox UI → Agent → the same Agentic API.

Proposed alignment with Case Manager: internal service-to-service ingress within the approved VNET; the PM chat backend terminates user authentication and derives tenant and operator context from its session. Veersa must confirm PM uses the same session and ingress model before this is adopted.

Hop	Identity and enforcement
Analyst → PM backend	Existing authenticated user session; backend checks PM permissions.
PM backend → Agent	Backend-generated contextToken; no tenant or scope derived from the user's message.
Agent → Agentic API	Relay contextToken verbatim in agreed transport metadata; Veersa validates tenant, resource access and requested operation.
Sandbox → Agent	X-Tenant-Id checked against a server-side tenant allow-list; simulation permission separately enforced.

Proposed decoded contextToken:

```
{
  "tenantId": "shcuat1b",
  "orgId": "dummy",
  "operatorId": "458272",
  "scope": ["policy:read", "history:read", "simulation:run"]
```

```
}
```

The Case Manager draft uses unsigned base64 JSON on restricted internal ingress. If reused here, only the trusted PM backend may supply product context; direct sandbox callers cannot supply it. Base64 is encoding, not authentication. Additional callers or a changed ingress boundary require an authenticated issuer mechanism, such as a signed token.

Thoughtful holds no analyst login token and makes no independent entitlement decisions. Veersa authorizes every data fetch, simulation submission and result retrieval. Any tenant mismatch is rejected. Conversation state is never an authorization credential.

3 Chat contract

POST /v1/chat, built by Thoughtful Labs. Reuse the shared chat envelope with surface set to policy_manager.

```
{
  "continuation": null,
  "conversationId": null,
  "history": null,
  "surface": "policy_manager",
  "message": "Recommend a policy for missed fraud and compare it with our current policies.",
  "context": {
    "caseId": null,
    "policySetId": "baseline-01",
    "scenarioId": "scenario-01"
  },
  "contextToken": "<backend-issued-context>",
  "options": {
    "runBacktest": true
  }
}
```

caseId is an optional investigation seed. It does not define the evaluation population. policySetId and scenarioId identify the baseline and historical scenario; they may be omitted only when the backend can resolve an unambiguous authorized default. Otherwise return needs_input with a follow-up question.

On the sandbox path, omit contextToken and provide tenantId; it must match the validated X-Tenant-Id. Product requests must not provide a separate tenantId. Every referenced case, policy set, scenario and simulation must belong to the authorized tenant.

options.runBacktest defaults to true for recommendation requests. It does not grant simulation permission. Case Manager's includeResolutions flag is not reused: PM requires historical outcomes, subject to the discovery/evaluation separation below.

Streaming response

Response is text/event-stream. Preserve the Case Manager event names.

Event	Payload	Cardinality
token	{delta}	0..n
tool_call	{id, name, args}	0..n; sanitized arguments
tool_result	{id, ms, rowCount, ok}	One per completed tool call; no raw rows
payload	Structured PM object below	Exactly one on a successfully completed turn
done	{conversationId, continuation, latencyMs, tokensIn, tokensOut}	Exactly one after payload
error	{code, message, retryable}	Terminal; no subsequent done

A turn can finish successfully with a pending simulation or insufficient data. done means the chat turn completed, not that the policy passed backtesting. Streamed prose must not claim an improvement before results are available.

Structured Policy Manager payload

```
{
  "answer": "The proposed refinement is ready for simulation.",
  "status": "candidate_ready",
  "recommendation": {
    "candidateId": "candidate-01",
    "version": 1,
    "changeType": "refine",
    "targetPolicyIds": ["policy-17"],
    "title": "Review high-risk transactions on new devices",
    "description": "Add a supported device signal to the existing condition.",
    "policySpec": {
      "format": "outseer-policy-dsl",
      "schemaVersion": "TBD",
      "definition": null
    },
    "validationStatus": "not_validated",
    "rationale": "Historical discovery evidence indicates a gap.",
    "evidenceRefs": ["evidence-01"],
    "assumptions": [],
    "limitations": ["Executable policy representation is pending agreement."]
  },
```

```
  "simulation": null,
  "citations": [
    {
      "id": "evidence-01",
      "source": "AgenticGetPolicyContext",
      "snapshotId": "discovery-01",
      "recordId": "segment-01",
      "fields": ["missedFraudCount"]
    }
  ],
  "followUpQuestions": [],
  "review": {
    "status": "not_submitted",
    "requiresSmeApproval": true
  }
}
```


The example intentionally has no executable definition. It must not be submitted until definition is populated and Veersa validates its format, fields, operators and actions.

status: needs_input, insufficient_data, candidate_ready, simulation_pending, evaluated or simulation_failed.

changeType: new, refine, threshold_update, merge or retire. For new, targetPolicyIds is empty; other operations identify the affected baseline policies. recommendation may be null when no supported candidate can be produced.

simulation, when present, includes simulationId, status, candidateId, candidateVersion, candidateHash, scenarioId, snapshotId, baseline reference, comparison and limitations. Completed comparisons use the simulation service's returned numbers unchanged. Every evidence reference resolves to a returned citation. Rationale without supporting data is explicitly an assumption.

No numerical model confidence is required in V0. Unknown values remain null with a reason; they are not converted to zero.

Errors

Use the shared Case Manager error envelope. Numeric codes are application codes; 4221 and 4222 are not HTTP statuses. Before streaming starts, map failures to an appropriate HTTP status. After streaming starts, emit error and close.

Code	Meaning	Retryable
401	Missing or invalid authentication/context credential	No
403	Tenant, scope or resource access	No

	denied	
404	Authorized resource not found	No
409	Candidate version, baseline version or idempotency conflict	No
422	Invalid request or unsupported policy definition	No
429	Rate or simulation quota limit	Yes, with backoff
503	Model, data or simulation service unavailable	Yes
4221	Continuation expired	No; start a new conversation
4222	Continuation invalid or tampered	No

A simulation failure after a candidate has been produced can return simulation_failed with the candidate and a safe failure reason. It must not be presented as an evaluated recommendation. Retries reuse the simulation idempotency key.

Conversation state

Retain history, continuation and conversationId for shared-client compatibility. First turn may omit all three. Proposed clarification to the Case Manager draft: subsequent turns supply exactly one; reject ambiguous combinations instead of silently applying precedence. history is restricted to evaluation callers.

Production continuation is opaque and sealed, or uses an authorized server-side conversation. Bind state to tenantId, orgId, operatorId and surface. Store candidate versions, scenario and baseline references, simulation IDs and evidence references; do not embed transaction datasets.

Veersa reauthorizes every turn. Thoughtful compacts the conversation and re-fetches stale live context. Immutable snapshots remain pinned. Changing the candidate, baseline or scenario invalidates the displayed comparison until an exact matching run is retrieved or a new run completes. Changing tenant resets the conversation.

4 Agentic data contract

Proposed GET /agentic/v1/policy-context?policySetId=baseline-01&scenarioId=scenario-01, built by Veersa. Optional caseId supplies a seed. Tenant identity comes from trusted context, not an unrestricted query parameter.

AgenticGetPolicyContext V0 is a normalized response over shared data services. The dedicated interface keeps new fields inside one contract. PM upstream routes, RPCs and database mappings have not been provided and must be confirmed by Veersa.

Response block	Required content
baseline	Policy-set ID, immutable version/hash, constituent policy IDs, executable definitions, priority and action resolution semantics
policySchema	Supported fields, types, units, operators, actions, DSL version and validation constraints
scenario	Scenario ID, historical window, timezone, cohort filters and evaluation unit
discovery	Discovery snapshot ID, missed-fraud/change-point summaries, outcome counts and resolvable evidence references
evaluation	Pinned evaluation snapshot reference, label cutoff, population counts and data-quality metadata; no held-out outcome rows exposed to the agent

constraints	Tenant-approved operating limits, such as maximum false-positive rate; absent limits explicitly marked unset
meta	Fetched time, source provenance, schema version, completeness, truncation and known limitations

The baseline must describe the policy set actually used for comparison. A single-rule improvement does not establish improvement in the full production policy set.

Field and outcome handling

Veersa normalizes dates to ISO 8601, documents timezone and window boundaries, and returns typed numeric values with units and currency. No display-label pairs or inferred units. PM units must be verified independently of the CM amount conversion.

Historical fraud labels are valid discovery inputs. Outcome-derived information—including retrospective fraud flags and back-coloring—cannot become a decision-time feature. Each permitted feature needs an availability definition that the simulator enforces.

Outseer must approve the mapping from source outcomes to fraud, genuine and unknown. Suspected/unknown/unresolved outcomes cannot silently become genuine. Unknown outcomes are excluded from the primary confusion matrix and counted separately.

Recommendation discovery and final evaluation use separate, identified data partitions. If the same evaluation set is inspected repeatedly during refinement, label results exploratory; reserve a separate holdout for final validation. No held-out outcome rows are returned through evidence retrieval.

Missing data, unavailable sources, truncation and insufficient label coverage must be visible to the agent and UI. A seed case alone cannot support a population-level uplift claim. Gold-layer availability is not assumed.

5 Simulation contract

Veersa executes the candidate and computes the metrics. Thoughtful calls the service and interprets its output. The model does not manufacture or independently recalculate the authoritative confusion matrix.

Submit

Proposed POST /agentic/v1/simulations. Require an Idempotency-Key scoped to tenant and normalized request.

```
{
  "candidate": {
    "candidateId": "candidate-01",
    "version": 1,
    "changeType": "refine",
    "targetPolicyIds": ["policy-17"],
    "policySpec": {
      "format": "outseer-policy-dsl",
      "schemaVersion": "1",
      "definition": "<validated executable definition>"
    }
  },
  "baseline": {
    "policySetId": "baseline-01",
    "version": "7"
  },
  "scenarioId": "scenario-01",
  "snapshotId": "evaluation-01",
  "comparisonMode": "baseline_vs_modified_policy_set"
}
```

This is an illustrative envelope, not an agreed DSL. The simulator validates the definition before execution and records its canonical hash. The candidate modifies a copy of the pinned baseline; it never mutates production. Veersa applies policy priority, overlap, action resolution and any required point-in-time feature history.

Return HTTP 202 with simulationId, status and pollAfterMs. Repeating the same key and request returns the same run; reusing the key for a different request returns 409.

Retrieve

Proposed GET /agentic/v1/simulations/{simulationId}. States: queued, running, completed, failed or cancelled. Result access requires tenant and scope checks even when the caller knows the run ID.

For queued/running, return status and pollAfterMs. For failed/cancelled, return a safe reason and retryability; no successful comparison. Completed results include immutable candidate/baseline/snapshot identity, engine version, start/end timestamps, data quality and the comparison.

Illustrative comparison for 10,000 labeled events:

```
{
  "evaluationUnit": "event",
  "positiveDefinition": "action mapped to positive under the approved scenario",
  "labelledCount": 10000,
  "excludedUnknownCount": 150,
  "baseline": {
    "tp": 80, "fp": 180, "tn": 9720, "fn": 20,
    "recall": 0.8,
    "precision": 0.307692,
```

```
    "falsePositiveRate": 0.018182
  },
  "candidate": {
    "tp": 90, "fp": 200, "tn": 9700, "fn": 10,
    "recall": 0.9,
    "precision": 0.310345,
    "falsePositiveRate": 0.020202
  },
  "delta": {
    "tp": 10, "fp": 20, "fn": -10,
    "recallPercentagePoints": 10,
    "falsePositiveRatePercentagePoints": 0.20202
  },
  "constraintCheck": {
    "status": "not_configured",
    "violations": []
  }
}
```

Metric semantics

Both runs use identical eligible records, outcome mapping and historical feature semantics. Specify event versus transaction evaluation and how actions map to a positive prediction before execution.

TP = fraud predicted positive; FP = genuine predicted positive; TN = genuine predicted negative; FN = fraud predicted negative. For each arm, TP + FP + TN + FN equals labelledCount. Fraud and genuine totals must match between arms.

Recall = TP / (TP + FN). Precision = TP / (TP + FP). False-positive rate = FP / (FP + TN). A zero denominator returns null with a reason. Rates are fractions in [0,1]; percentage-point deltas are explicitly named.

“Fraud capture” means labeled-fraud detection recall here. Historical replay does not establish realized fraud prevention. Monetary capture or net impact requires amounts, currency handling and an agreed cost model; otherwise return null. Do not infer prevented losses from flags.

An improvement is a measured trade-off, not an approval. Return constraintCheck as pass, fail or not_configured. An unset false-positive tolerance cannot produce a passing result.

Long-running work and reuse

Do not hold a chat stream open for an unbounded simulation. If the agreed per-turn wait budget is exceeded, return simulation_pending with the run ID and finish the turn. The next turn retrieves the existing run through the agent; UI refresh behavior must be agreed.

Reuse only completed runs matching tenant, candidate hash, baseline version, snapshot, engine version and evaluation settings. A chat disconnection does not resubmit or cancel a run. Retry limits, concurrency, maximum candidate attempts and execution deadlines are server-enforced configuration.

6 Sandbox and review

Extend the existing two-surface sandbox. The PM tab includes tenant, baseline and historical scenario selectors, an optional case seed, streamed chat, candidate policy, supporting evidence and baseline/candidate metrics.

Show candidate version and simulation status together so old metrics cannot appear under an edited candidate. Show missing labels, exploratory evaluation and failed constraints beside the results. The trace panel shows sanitized tool arguments, timing, row counts, model/endpoint identity and run IDs; restricted raw model output is development-only.

Veersa owns approve, reject and request-modification actions in the PM backend. Approval records the operator, timestamp, candidate hash/version, baseline version, snapshot and simulation ID. Changing the candidate requires a new review. No chat response can mark a recommendation approved. Any eventual deployment remains a separate approved workflow.

Audit records correlate request, conversation, tenant, operator, model/harness version, candidate, data snapshots, simulation and SME decision. Do not log context credentials or unrestricted historical rows.

7 V0 acceptance and open items

Proposed V0 acceptance: one authorized tenant, one supported policy representation, one pinned baseline/scenario and one complete recommendation → backtest → explanation flow. Fixtures may establish schemas before live data is ready; they must be labeled fixtures. No PM sprint dates are committed by this draft.

Acceptance requires tenant isolation, executable-policy validation, an independently checked fixture confusion matrix, idempotent submission, version-matched results, pending/failure handling, resolvable evidence and no production policy writes.

Item	Proposed position	Confirmation owner
Orchestration	Agent owns the tool loop; Veersa owns execution	Thoughtful + Veersa
PM auth topology	Reuse CM pattern only after PM backend verification	Veersa
Executable policy format	Versioned Outseer-supported	Veersa + Outseer

	representation; exact grammar, actions and validation errors TBD	
Upstream field mapping	Map each context field to a real service/data source	Veersa
Baseline semantics	Pin full policy set, priority and action resolution	Veersa + Outseer
Evaluation semantics	Confirm evaluation unit, positive action mapping, labels and cutoffs	Outseer + Veersa
Data readiness	Discovery and evaluation snapshots with point-in-time features	Veersa + Outseer
Business constraints	Agree acceptable friction and any monetary cost model	Outseer
Long-running jobs	Agree poll interval, chat wait budget, deadlines and UI refresh	Thoughtful + Veersa
Continuation	Agree shared-client validation and production state mechanism	Thoughtful + Veersa
Review persistence	PM backend owns SME actions; final route/storage TBD	Veersa
Delivery and latency	Set milestones and targets after PM data/engine sizing; do	All parties

	not inherit CM latency targets	

Source basis: supplied “Agentic Interface Contract — Case Manager”, Draft 02, and “Outseer Agentic AI PoC” PRD. This draft preserves their ownership boundaries and extends the interfaces for the PM recommendation and backtesting workflow.